



การใช้ EFFECT และ CONTEXT

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

การตอบสนองต่อ Effect

ใน React นั้น มีลักษณะบางอย่างที่ถือว่าเป็นผลกระทบ (Effect) ต่อคอมโพเนนต์ เช่น การแสดงผลครั้งแรก, การแสดงผลซ้ำ (re-render), การเปลี่ยนแปลงค่าใน State, การรับส่งข้อมูลกับเซิร์ฟเวอร์ เป็นต้น นอกจากนี้ก็ยังมีลักษณะอื่นๆ อีกมากมายที่จัดว่าเป็น Effect ซึ่งในบางกรณี เราอาจต้องการตอบสนองหรือกำหนดการกระทำบางอย่างเมื่อมี Effect เกิดขึ้น แต่จะใช้วิธีการที่แตกต่างกันระหว่าง Function Component และ Class Component ดังรายละเอียดต่อไปนี้

- **การเกิด Effect กับ Function Component**

ในกรณีของ Function Component จะมีฟังก์ชันในกลุ่ม React Hook สำหรับดำเนินการเมื่อเกิด Effect มาให้เราใช้งานโดยตรงอยู่แล้ว นั่นคือ `useEffect()` โดยมีหลักการดังนี้

- เราอาจนำเข้าฟังก์ชัน `useEffect()` โดยตรง หรือจะเรียกผ่านคอมโพเนนต์ React ก็ได้
- การกระทำหรือการตอบสนองเมื่อเกิด Effect เราจะกำหนดในแบบ callback ให้แก่ฟังก์ชัน `useEffect()` ดังรูปแบบต่อไปนี้

```
useEffect(callback [, dependencies])
```

การตอบสนองต่อ Effect

```
import React, { useEffect } from 'react'

...

function MyComponent() {
  useEffect(function() {
    //สิ่งที่จะดำเนินการเมื่อเกิด Effect
  })

  /* หรือวิธีอื่นๆ เช่น
  React.useEffect(() => { //ใช้ Arrow Function
    //สิ่งที่จะดำเนินการเมื่อเกิด Effect
  })
  */

  return (...)
}
```

การตอบสนองต่อ Effect

- ต่อไปเราต้องพิจารณาว่า สิ่งที่มีผลต่อการเกิด Effect หรือ Dependencies หรือกล่าวอีกอย่างคือเราจะเรียก callback ที่กำหนดให้แก่ useEffect() ขึ้นมาทำงานเมื่อใด นั่นเอง ซึ่งมีทางเลือกดังนี้
 - ถ้าต้องการให้เรียก callback เมื่อเกิด Effect ในทุกกรณี (เช่น แสดงคอมโพเนนต์, re-render, ติดต่อกับเซิร์ฟเวอร์, ฯลฯ) ก็ไม่ต้องระบุ dependencies เช่น

```
React.useEffect() => {  
    //...  
}
```

- ถ้าต้องการให้เรียก callback เฉพาะ **ครั้งแรก** ที่แสดงคอมโพเนนต์เพียงครั้งเดียวเท่านั้น ให้กำหนด dependencies เป็นอาร์เรย์ว่าง เช่น

```
React.useEffect() => {  
    //...  
}, [] )    //กำหนดอาร์เรย์ว่าง
```

การตอบสนองต่อ Effect

- ๑ ถ้าต้องการให้เรียก callback เมื่อค่าตัวแปร State ตัวใดตัวหนึ่งเปลี่ยนแปลงไป ก็ให้ระบุชื่อตัวแปรเหล่านั้นไว้ในอาร์เรย์ (กำหนดได้มากกว่า 1 ตัว) เช่น

```
let [a, setA] = React.useState()
let [b, setB] = React.useState()

React.useEffect(() => {
  //...
}, [a, b]) //ให้เรียก callback เมื่อ a หรือ b เปลี่ยนแปลง
```

- ฟังก์ชันที่เป็น callback จะถูกเรียกหลังจากที่แสดงผลคอมโพเนนต์รวมถึง re-render ไปแล้ว

การตอบสนองต่อ Effect

react/app1/src/effect.js

```
import React, { ..... } from 'react'

function Effect() {
  const [count, .....]=useState(.....)
  const [name, .....]=useState(".....")

  useEffect(()=>{
    document.title=`.....`
  }, [.....])

  return (
    <center>
      <h1>{.....}</h1>
      <h3>{name}</h3>
```

```
      <button onClick={()=>setCount(count+1)}>
        Increase</button>
      <button onClick={()=>.....}>
        Decrease</button>
      <button onClick={()=>setName("Jack Wick")}>
        .....</button>
    </center>
  )
}

export default Effect
```

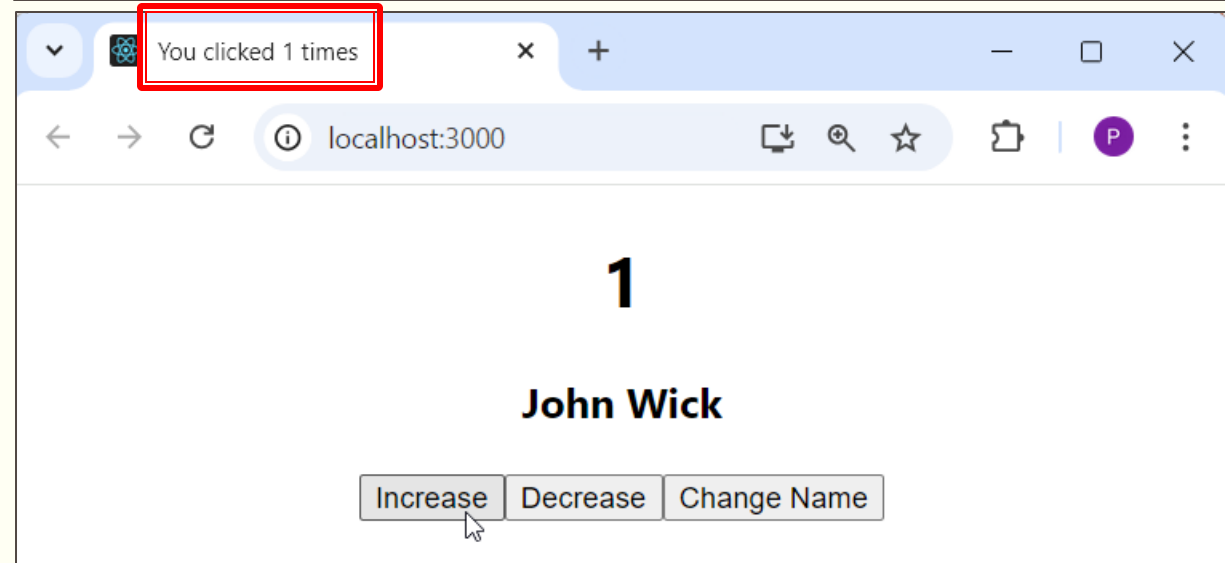
การตอบสนองต่อ Effect

react/app1/src/App.js

```
import React from 'react'
import ..... from './effect';

function App() {
  return .....
}

export default App;
```



การตอบสนองต่อ Effect

- การเกิด Effect กับ Class Component

ในกรณีของ Class Component จะไม่มีเมธอดสำหรับจัดการ Effect โดยตรง ก็ยังมีลักษณะที่พอจะเทียบเท่ากันได้ นั่นคือเมธอด `componentDidMount()` และ/หรือ `componentDidUpdate()` ดังหลักการต่อไปนี้

- หากเราจะทำสิ่งนั้น **เพียงครั้งเดียว** หลังจากที่แสดงผลคอมโพเนนต์ไปแล้ว ก็ให้โอเวอร์ไรด์หรือกำหนดการกระทำไว้ในเมธอด `componentDidMount()`

```
class MyComponent extends React.Component {  
    ...  
    render() {  
        return ...  
    }  
  
    componentDidMount() {  
        //กำหนดการกระทำหลังจากแสดงผลคอมโพเนนต์ครั้งแรก  
    }  
}
```

การตอบสนองต่อ Effect

- หากเราจะทำสิ่งนั้นเฉพาะเมื่อคอมโพเนนต์ re-render ไปแล้วเท่านั้น (มักเกิดจากค่าใน State เปลี่ยนแปลงไป) ก็ให้โอเวอร์ไรด์เมธอด `componentDidUpdate()`

```
class MyComponent extends React.Component {  
  ...  
  render() {  
    return ...  
  }  
  
  componentDidUpdate() {  
    //กำหนดการกระทำหลังจากคอมโพเนนต์ re-render  
  }  
}
```

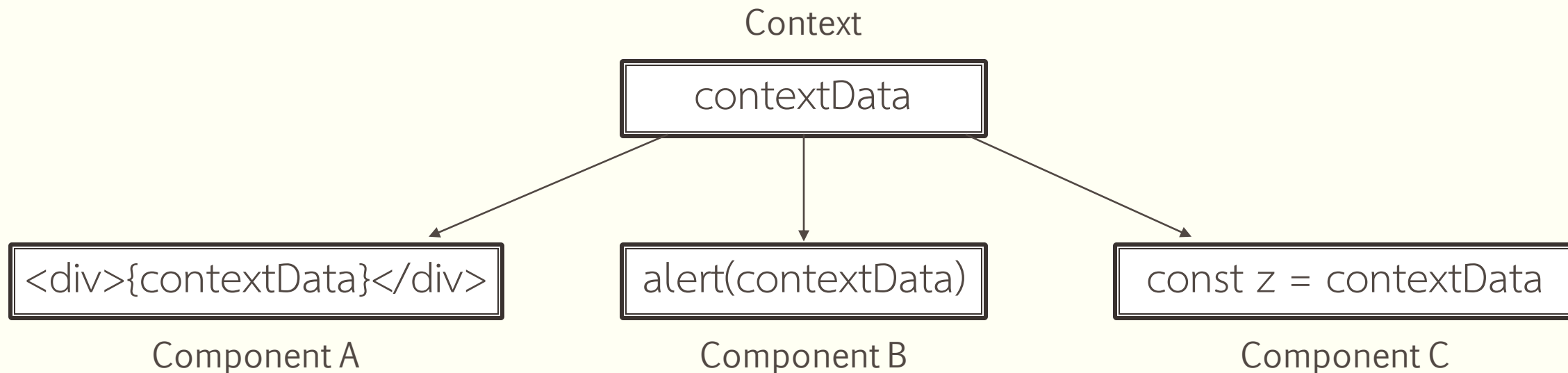
การตอบสนองต่อ Effect

- หากเราจะทำสิ่งนั้นทั้งกรณีที่แสดงผลคอมโพเนนต์ครั้งแรก และกรณีที่คอมโพเนนต์ re-render ก็ให้โอเวอร์ไรด์ร่วมกันทั้งเมธอด `componentDidMount()` และ `componentDidUpdate()`

```
class MyComponent extends React.Component {  
  ...  
  render() {  
    return ...  
  }  
  
  componentDidMount() {  
    //กำหนดการกระทำหลังจากแสดงผลคอมโพเนนต์ครั้งแรก  
  }  
  
  componentDidUpdate() {  
    //กำหนดการกระทำหลังจากคอมโพเนนต์ re-render  
  }  
}
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

การสร้างเว็บเพจในแบบของ React นั้น จะเป็นการนำคอมโพเนนต์ย่อยๆ ของแต่ละส่วนมารวมเข้าด้วยกัน จากเหตุผลดังกล่าว จึงอาจมีบางกรณีที่ต้องใช้ข้อมูลบางอย่างร่วมกันระหว่างคอมโพเนนต์ อย่างไรก็ตาม เราไม่สามารถเข้าถึงข้อมูลที่อยู่ต่างคอมโพเนนต์โดยตรงได้ แต่ต้องใช้วิธีการสร้างข้อมูลส่วนกลางที่เรียกว่า Context เพื่อให้คอมโพเนนต์ต่างๆ สามารถใช้งานร่วมกันได้ ดังหลักการในภาพถัดไป



การจัดเก็บข้อมูลส่วนกลางด้วย Context

- การสร้างและกำหนด Provider สำหรับ Context

เนื่องจากข้อมูลแบบ Context จะต้องใช้งานร่วมกันระหว่างคอมโพเนนต์ ดังนั้น เราควรแยกมาสร้างเป็นไฟล์ไว้ต่างหากในแบบโมดูล แล้วค่อยนำเข้าไปในคอมโพเนนต์ที่ต้องการใช้งาน และการกำหนดค่าให้กับ Context นั้น ต้องดำเนินการผ่าน Provider ดังแนวทางต่อไปนี้

- สร้างไฟล์เพิ่มเข้าในแอป เช่น ในที่นี้กำหนดชื่อเป็น context.js โดยตัวแปรที่จะเก็บข้อมูล Context นั้น ต้องกำหนดค่าด้วยฟังก์ชัน createContext() ซึ่งจะนำเข้าโดยตรง หรือจะเรียกผ่านคอมโพเนนต์ React ก็ได้ เช่น

```
react/app1/src/context.js
```

```
import React, { createContext } from 'react'
```

```
export const userContext = createContext() //หรือ React.createContext()
```

```
/* เนื่องจากเป็นตัวแปร ดังนั้น จึงส่งออกแบบ default ไม่ได้ */
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

- ตัวแปรหรือคอมโพเนนต์แบบ Context ที่เราสร้างขึ้นจะมีพร็อพเพอร์ตี้ Provider สำหรับใช้ในการจัดหาหรือกำหนดค่าให้กับ Context ซึ่งโดยส่วนใหญ่ เรามักจะวาง Provider ไว้ในไฟล์ App.js โดยนำเข้า Context จากไฟล์ที่สร้างเอาไว้ แล้วห่อหุ้มคอมโพเนนต์ทั้งหมดที่จะนำค่าจาก Context ไปใช้งานด้วย Provider แต่ถึงคอมโพเนนต์นั้นจะไม่ใช้ค่าจาก Context ก็ห่อหุ้มด้วย Provider ได้ เช่น สมมติว่าภายในแอปเรามี 2 คอมโพเนนต์คือ Header และ Content เราก็ห่อหุ้มด้วย Provider พร้อมกำหนดค่าผ่านพร็อพเพอร์ตี้ value ดังนี้

react/app1/src/App.js

```
import React from 'react'
import { useContext } from 'react' //ต้องนำเข้า Context
import Header from './context-header'
import Content from './context-content'
```

```
export default function App(){
  return (
    <userContext.Provider value={'Tom Jerry'}>
      <Header/>
      <Content/>
    </userContext.Provider>
  )
}
```

จากโค้ด Tom Jerry คือค่าที่กำหนดให้แก่ Context ซึ่งจะนำไปใช้ในคอมโพเนนต์ต่างๆ

การจัดเก็บข้อมูลส่วนกลางด้วย Context

- การใช้ข้อมูล Context ใน Class Component

คอมโพเนนต์ที่จะนำค่าจาก Context มาใช้งานได้ ต้องห่อหุ้มด้วย Provider เอาไว้แล้ว (ตั้งในไฟล์ App.js) ซึ่งหากเป็น Class Component การเข้าถึงค่าใน Context ก็มีวิธีการดังนี้

- นำเข้าตัวแปรหรือคอมโพเนนต์ของ Context ที่ได้สร้างเอาไว้แล้ว
- ต้องนำตัวแปร Context มากำหนดให้แก่พร็อพเพอร์ตี้ **contextType** ในแบบ static ซึ่งพร็อพเพอร์ตี้นี้เป็นของ React Component อยู่แล้ว เช่น

```
import { useContext } from './context'

export default class Header extends React.Component {
  static contextType = useContext
  ...
}
```

- ต่อไป การอ้างถึงค่าจาก Context ให้อ่านจากพร็อพเพอร์ตี้ที่ชื่อ **context** เช่น this.context ดังนั้นลักษณะโดยรวมของ Class Component กรณีที่ใช้ค่าจาก Context จะเป็นดังนี้

การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/context-header.js

```
import React from 'react'

import { useContext } from './context'

export default class Header extends React.Component {
  static contextType = useContext

  render() {
    const user = this.context

    const headerStyle = {
      backgroundColor: '#cee',
      textAlign: 'center',
      padding: 5
    }
  }
}
```

```
return (  
    <div style={headerStyle}>  
        <a href="/">Home</a>&nbsp;-&nbsp;  
        <a href="/product">Product</a>&nbsp;-&nbsp;  
        <a href="/contact-us">Contact Us</a>&nbsp;-&nbsp;-&nbsp;  
        [{user}&nbsp;-&nbsp;-&nbsp;<a href="/logout">Signout</a>]  
    </div>  
)
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/context.js

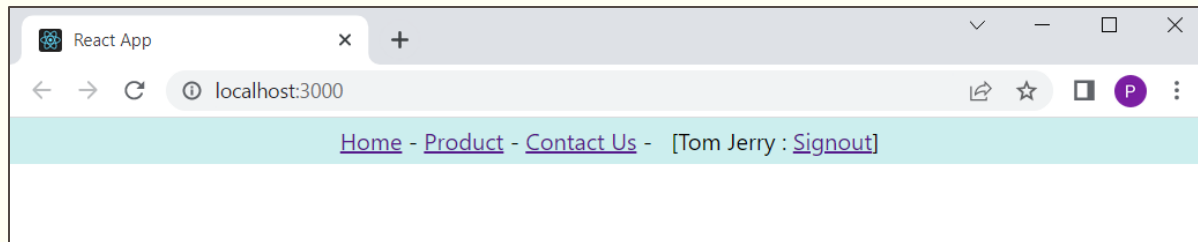
```
import React, {createContext} from 'react'

export const userContext = createContext()
```

react/app1/src/App.js

```
import React from 'react'
import { userContext } from './context'
import Header from './context-header'

export default function App() {
  return (
    <userContext.Provider value={'Tom Jerry'}>
      <Header/>
    </userContext.Provider>
  )
}
```



การจัดเก็บข้อมูลส่วนกลางด้วย Context

- การใช้ข้อมูล Context ใน Function Component

การนำค่าจาก Context มาใช้ใน Function Component สามารถทำได้ในแบบง่ายๆ เพราะมีฟังก์ชันในกลุ่ม React Hook ให้ใช้งานโดยตรงอยู่แล้ว นั่นคือ useContext() ดังแนวทางต่อไปนี้

react/app1/src/context-content.js

```
import React from 'react'
import { useContext } from './context'

export default function Content() {
  let user = React.useContext(userContext)

  const contentStyle = {
    backgroundColor: '#ddd',
    textAlign: 'center',
```

```
    margin: 10,
    padding: 10
  }

  return (
    <div style={contentStyle}>
      Hello {user}
    </div>
  )
}
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/context.js

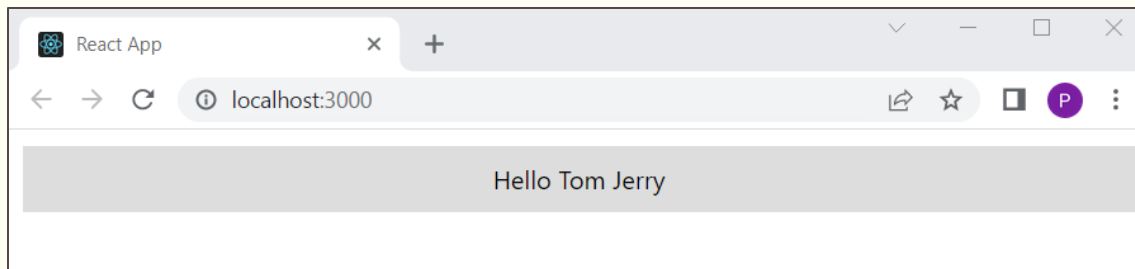
```
import React, {createContext} from 'react'

export const userContext = createContext()
```

react/app1/src/App.js

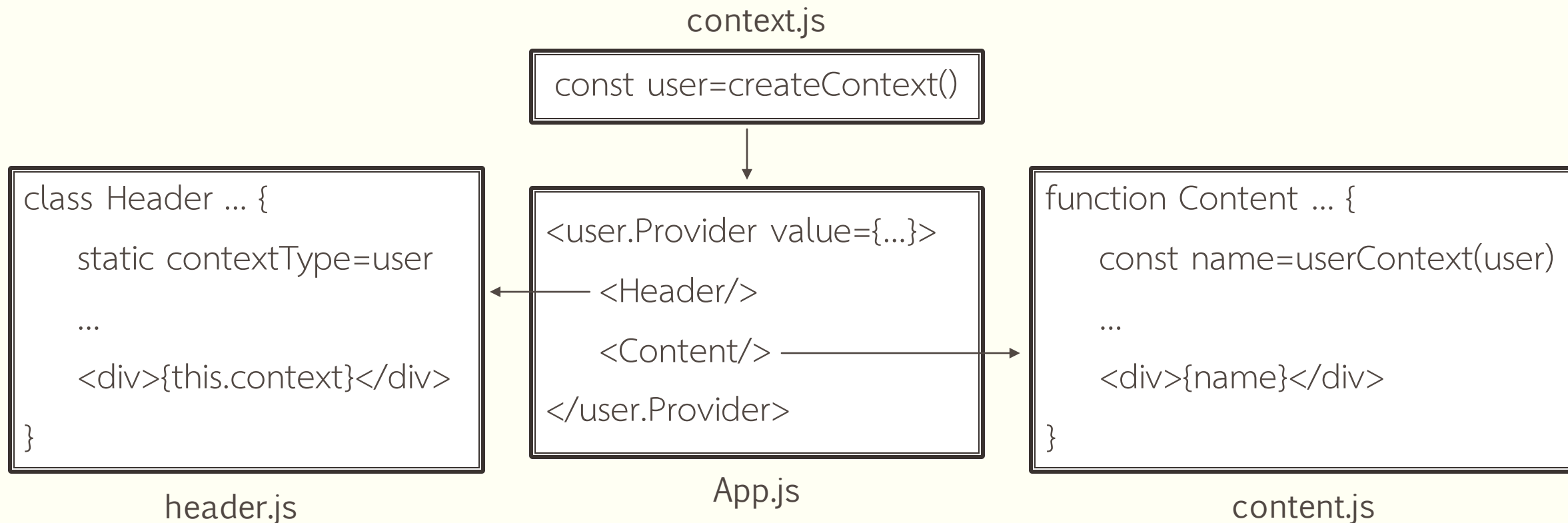
```
import React from 'react'
import { userContext } from './context'
import Content from './context-content'

export default function App() {
  return (
    <userContext.Provider value={'Tom Jerry'}>
      <Content/>
    </userContext.Provider>
  )
}
```



การจัดเก็บข้อมูลส่วนกลางด้วย Context

จากขั้นตอนสร้างทั้งหมดที่กล่าวมา นับตั้งแต่การสร้างตัวแปรแบบ Context การห่อหุ้มด้วย Provider รวมถึงการนำมาใช้งานทั้งใน Class และ Function Component สามารถสรุปได้ดังแผนภาพด้านล่าง ทั้งนี้หากกำหนดโค้ดดังที่ผ่านมา เมื่อรวมทั้งหมดเข้าด้วยกันแล้วทำการทดสอบก็จะได้ผลดังภาพต่อไปนี้

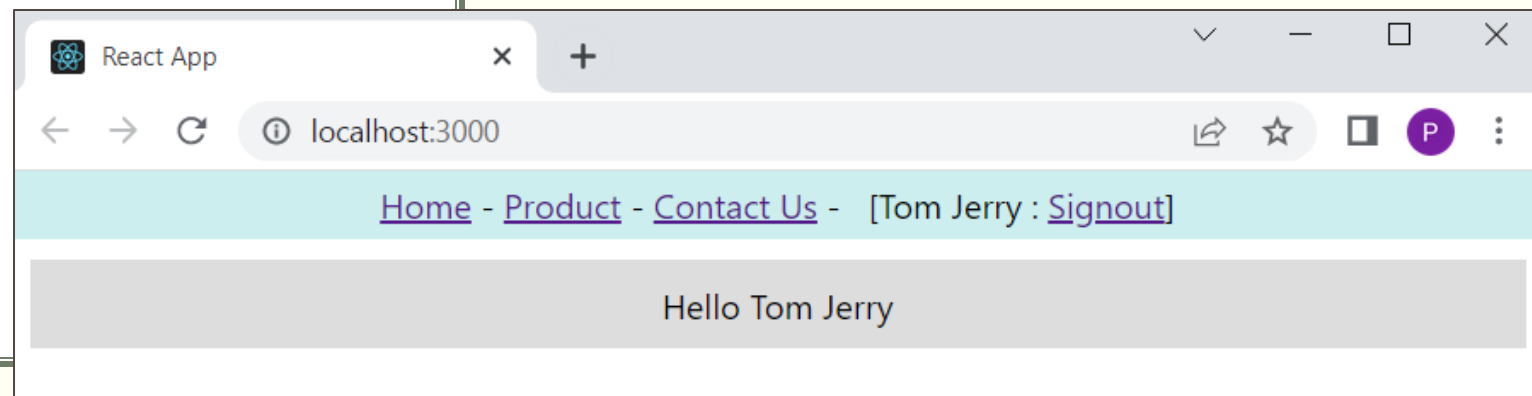


การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/App.js

```
import React from 'react'
import { useContext } from './context'
import Header from './context-header'
import Content from './context-content'

export default function App() {
  return (
    <userContext.Provider value={'Tom Jerry'}>
      <Header/>
      <Content/>
    </userContext.Provider>
  )
}
```



การจัดเก็บข้อมูลส่วนกลางด้วย Context

- การจัดเก็บข้อมูลแบบ State ไว้ใน Context

การใช้ข้อมูลจาก Context เรากำหนดค่าของมันแบบตายตัวไว้ที่ Provider จึงไม่สามารถอัปเดตข้อมูลดังกล่าวในภายหลังให้สอดคล้องกันทุกคอมโพเนนต์ได้ ซึ่งเราอาจแก้ปัญหานี้โดยการจัดเก็บข้อมูลแบบ State ลงใน Context แทนรูปแบบเดิม แล้วต่อไปเมื่อเราอัปเดตข้อมูล Context ที่คอมโพเนนต์ใด ก็จะส่งผลไปถึงคอมโพเนนต์อื่นๆ ทันที ซึ่งมีหลักการดังต่อไปนี้

- ที่ไฟล์การสร้าง Context ก็กำหนดเหมือนเดิม เช่น

```
react/app1/src/context.js
```

```
import React, { createContext } from 'react'
```

```
export const userContext=createContext() //หรือ React.createContext()
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

- ที่ไฟล์ App.js ซึ่งโดยปกติจะอยู่ในรูปแบบ Function Component ดังนั้น เราก็สร้าง State ตามหลักการของเดิม เช่น `let [user, setUser]=React.useState()`
- สร้าง Provider แล้วนำค่าจาก State มากำหนดให้แก่พร็อพเพอร์ตี้ `value` ในแบบอาร์เรย์ทั้งค่าของมันและฟังก์ชันสำหรับเปลี่ยนค่า เช่น

react/app1/src/App.js

```
import React from 'react'
import { userContext } from './context'
import ..... from './context-header2'
import ..... from './context-content2'
```

```
export default function App() {
  let [user, setUser] = .....
  return (
    <..... value={[user, setUser]}>
      <Header2/>
      <Content2/>
    </.....>
  )
}
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

- ที่คอมโพเนนต์ซึ่งเราจะนำค่าจาก Context ไปใช้งาน จะต้องอ่านค่าของ Context ในแบบ Array Destructuring เพื่อให้ได้ทั้งตัวแปรและฟังก์ชันในการเปลี่ยนค่าของ State จากนั้น เราก็นำจากค่าตัวแปรไปใช้งานในคอมโพเนนต์ตามปกติ และถ้าจะอัปเดตก็ทำผ่านฟังก์ชันของ State ดังแนวทางต่อไปนี้

react/app1/src/context-header2.js

```
import React from 'react'
import {.....} from './context'

export default class ..... extends React.Component {
  static contextType = useContext

  render() {
    let [user, setUser] = .....
```

```
const headerStyle = {
  backgroundColor: '#cee',
  textAlign: 'center',
  padding: 5
}

const onClickSignout = (event) => {
  event.preventDefault()
  setUser('')
}
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/context-header2.js (ต่อ)

[illegible]

```
{
  (user)
  ? <span>[{user}&nbsp;&nbsp;<a href=" "
    onClick={.....}>Signout</a>]</span>
  : <span>[<a href=" " onClick={.....}>
    Signin</a>]</span>

}
</div>
)
}
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

react/app1/src/context-content2.js (ต่อ)

```
import ..... from 'react'
import { ..... } from './context'

export default function ..... {
  let [user, setUser] = .....

  const ..... = {
    backgroundColor: '#ddd',
    textAlign: 'center',
    margin: 10,
    padding: 10
  }
```

```
const onClickSignin = (.....) => {
  event.preventDefault()
  setUser('.....')
}

return (
  <div style={contentStyle}>
    {
      (user)
      ? <span>Hello {.....}</span>
      : <span>Please <a href=" " onClick={.....}>
          Signin</a></span>
    }
  </div>
)
```

การจัดเก็บข้อมูลส่วนกลางด้วย Context

